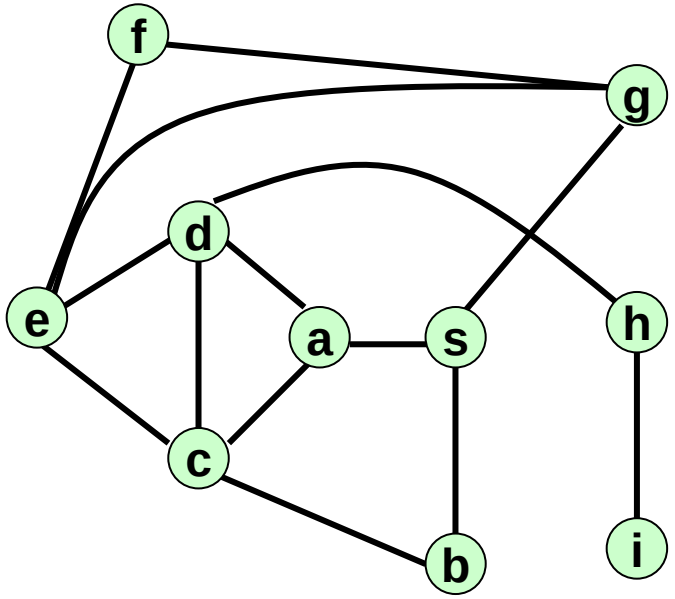
Busca em Largura

Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s)
  marcar(s)
  enquanto Q é não vazia
     se w ∈ A(v) não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
        se (v,w) não visitada
            visitar(v,w)
```

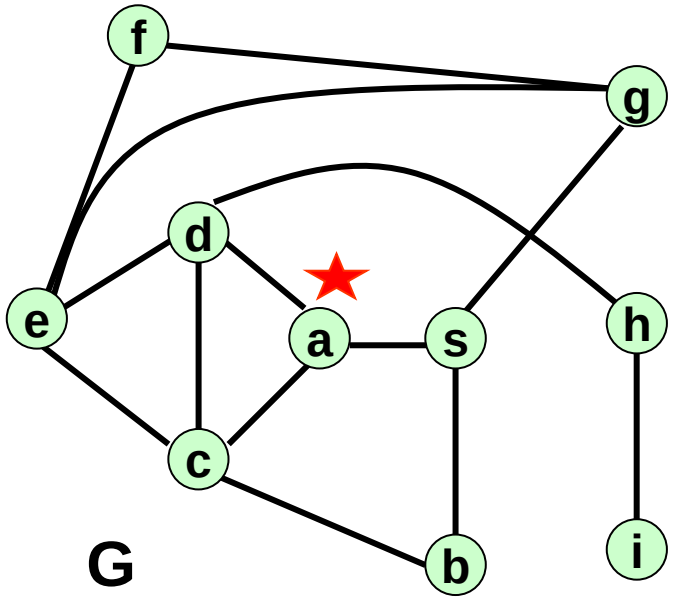
Algoritmo



G

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
        se (v,w) não visitada
             visitar(v,w)
```

Algoritmo

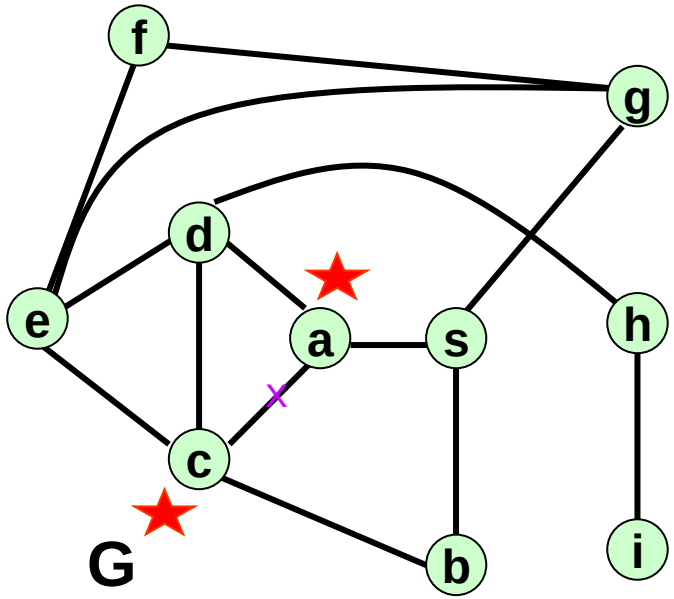


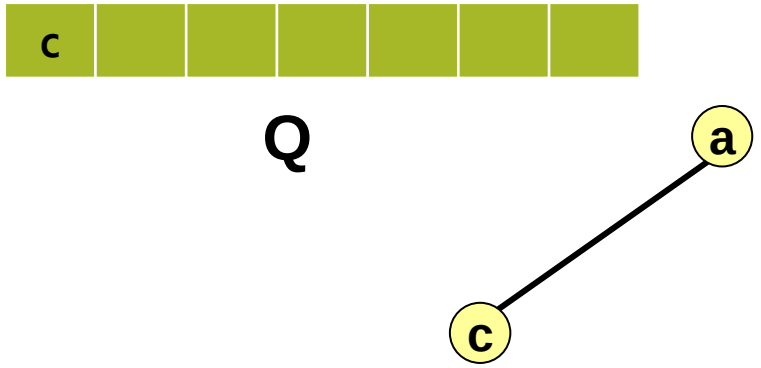


Q

a

Algoritmo

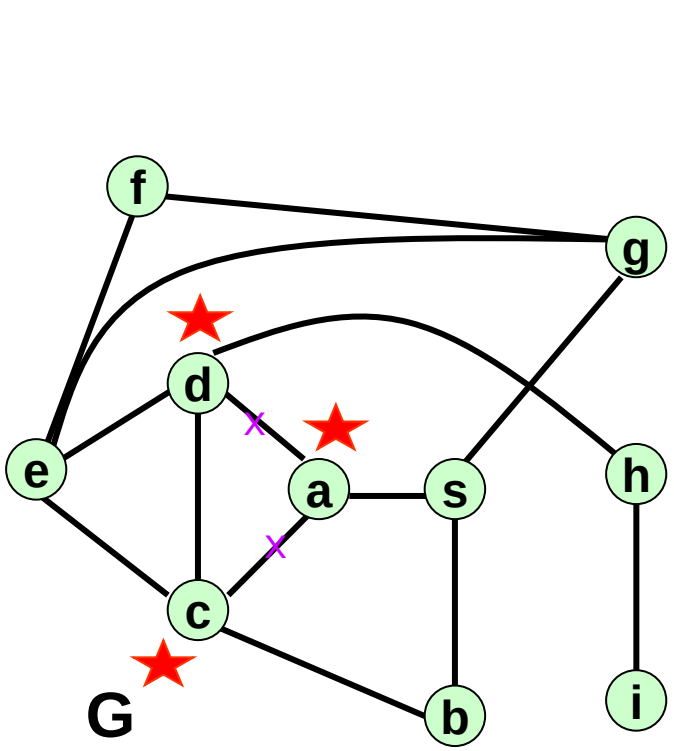





```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
            visitar(v,w)
```

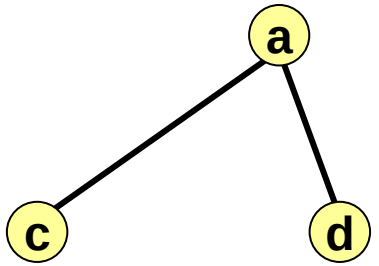
Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
    v ← remove_fila(Q)
    para w ∈ A(v) efetuar
    se w não é marcado então
        insere_fila(Q,w); marcar(w); visitar(v,w)
    senão
      se (v,w) não visitada
          visitar(v,w)
```





Q

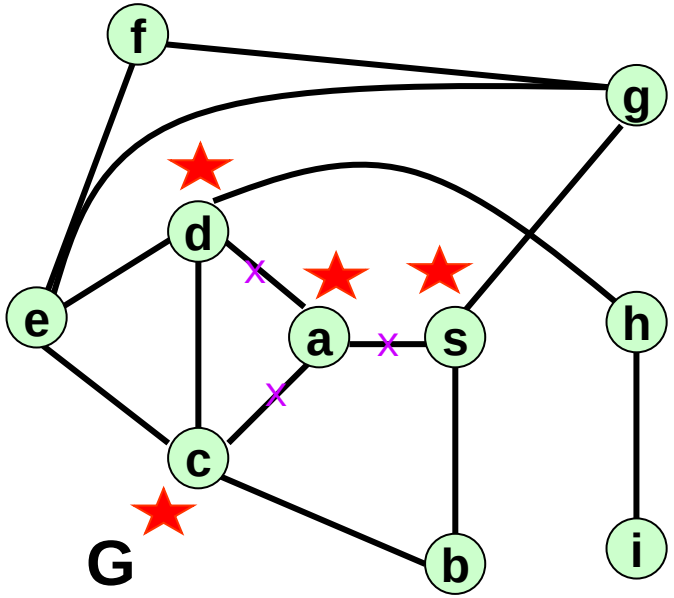


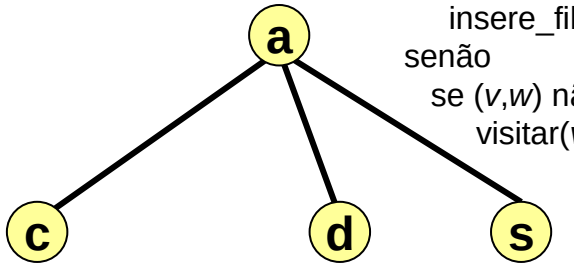
Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



Q



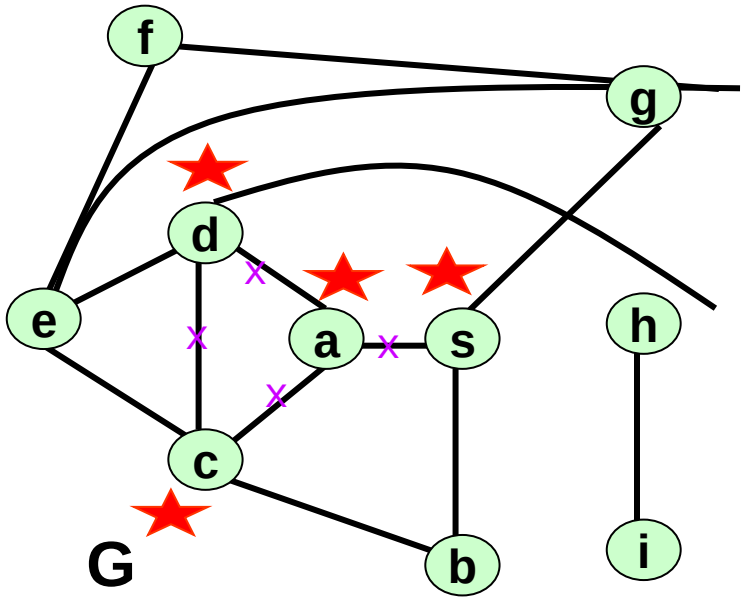


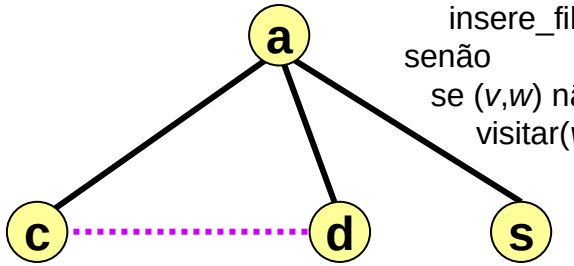
Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



Q

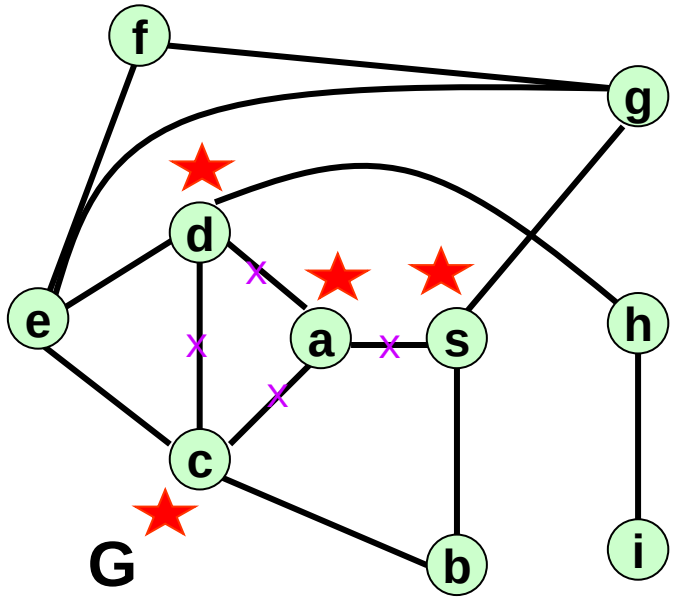


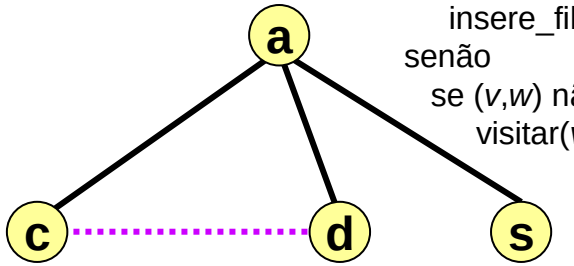


Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```





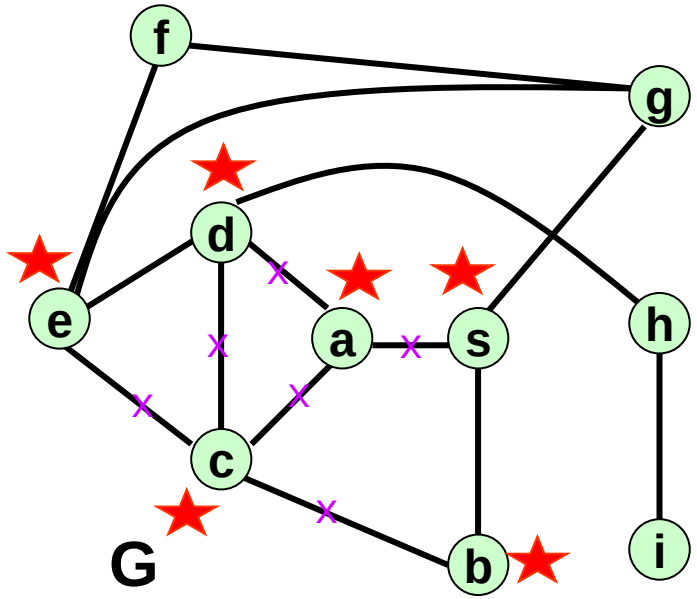


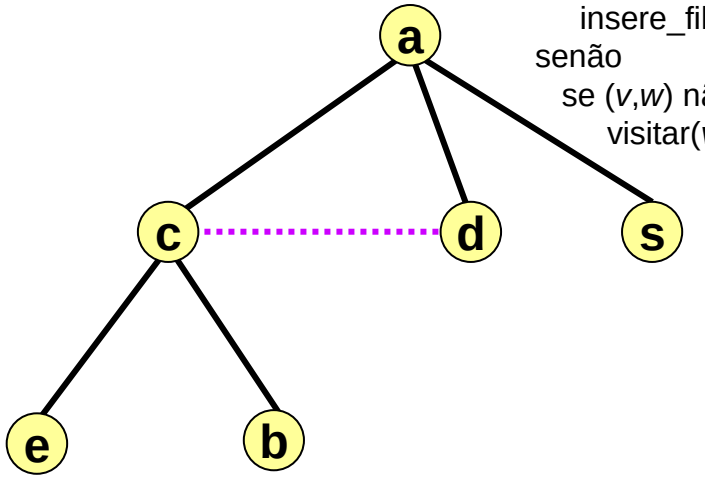
Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



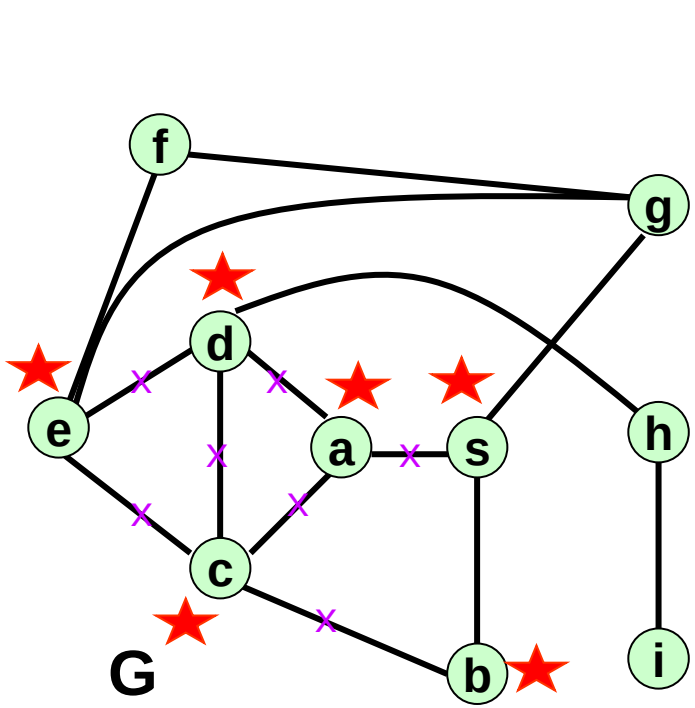
Q



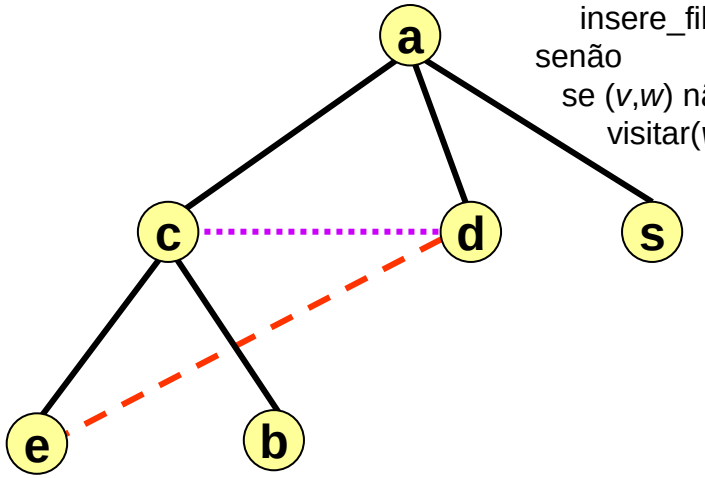


Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```

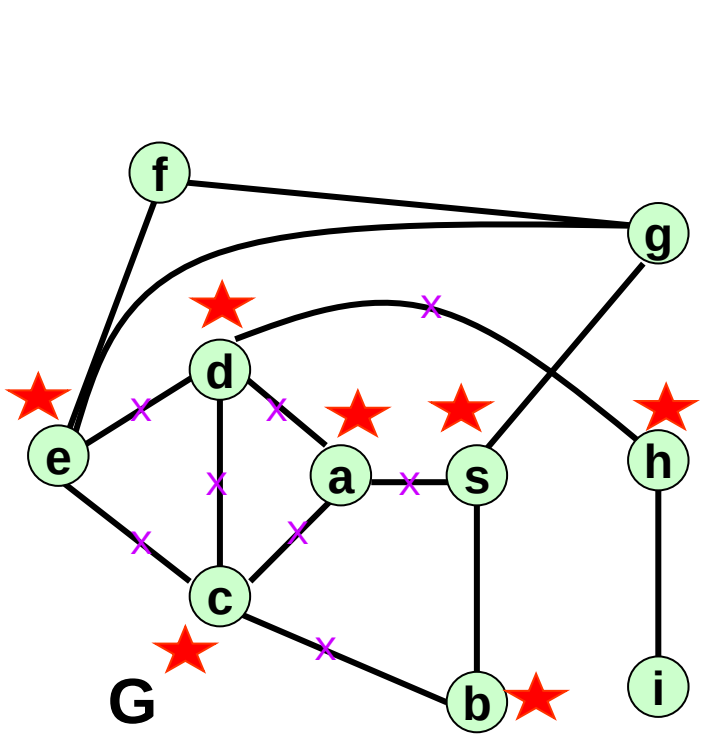




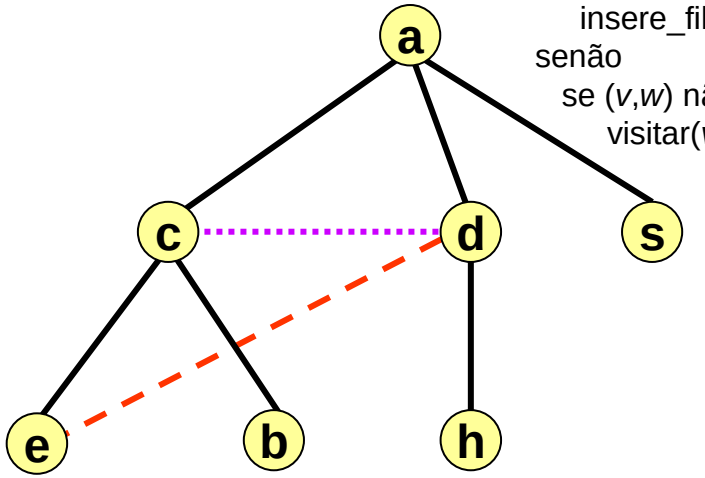


Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```

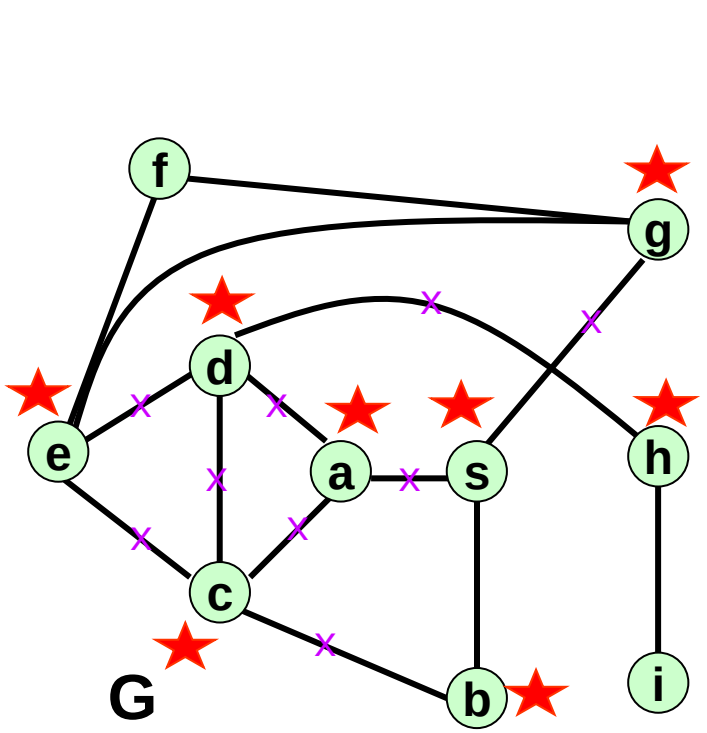






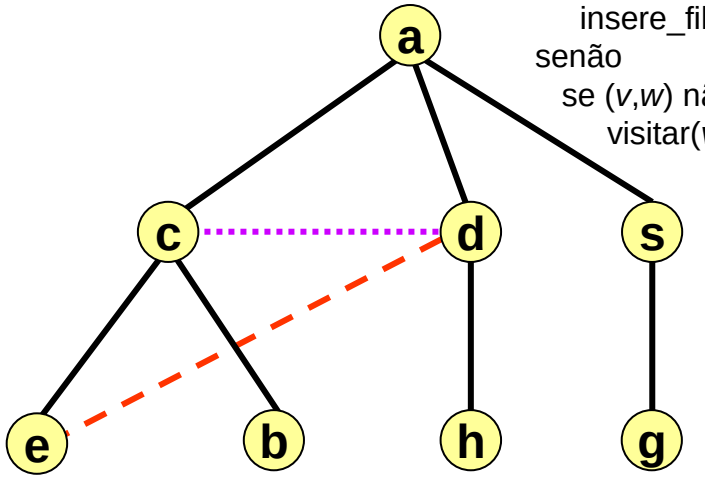
Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



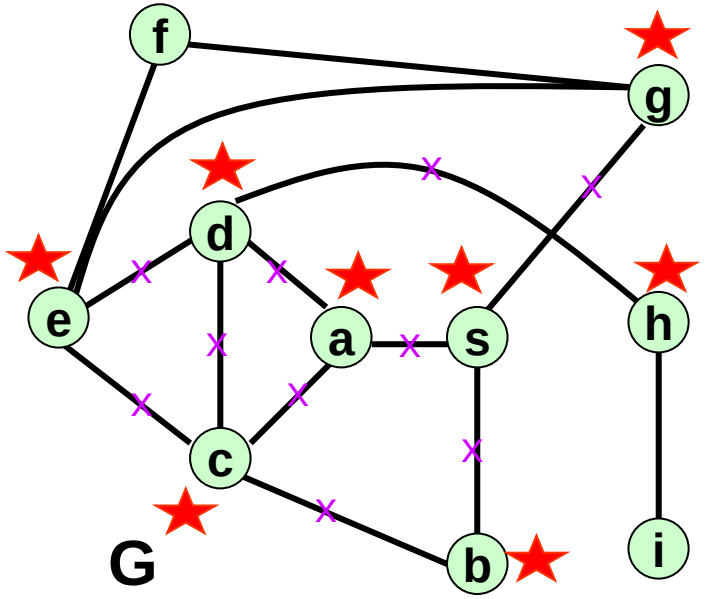


Q



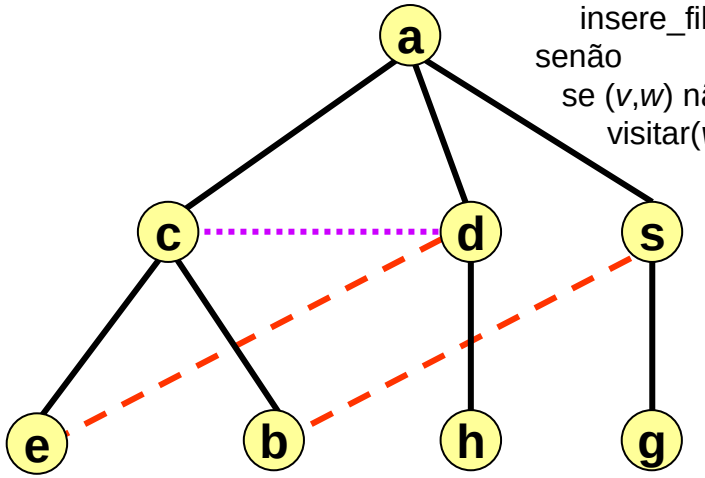
Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



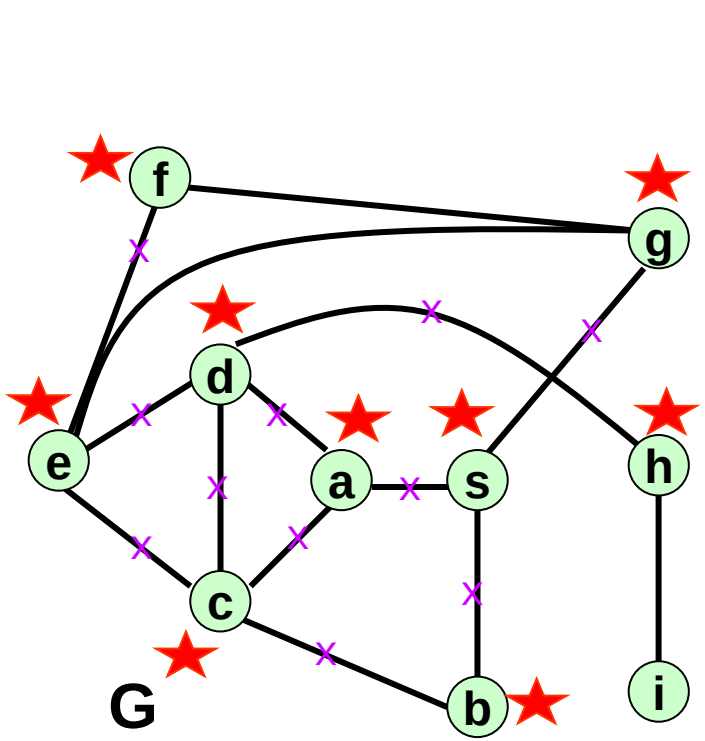


Q



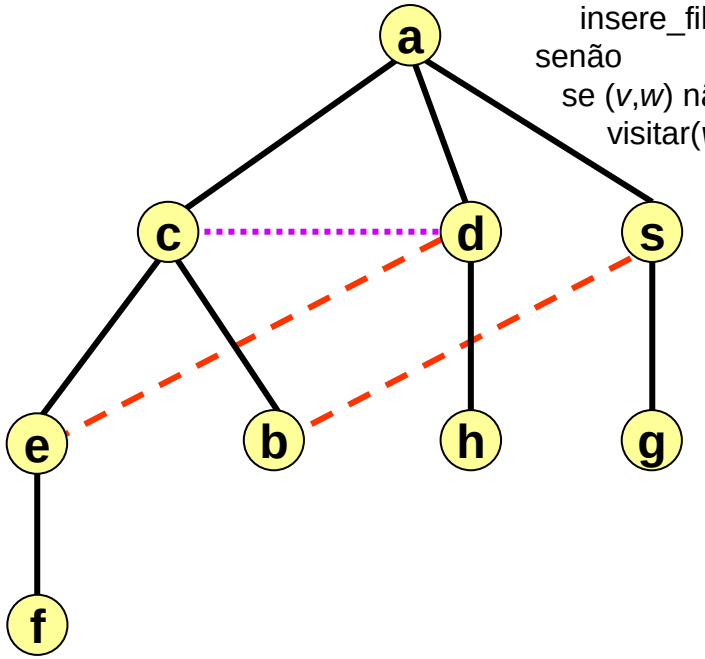
Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



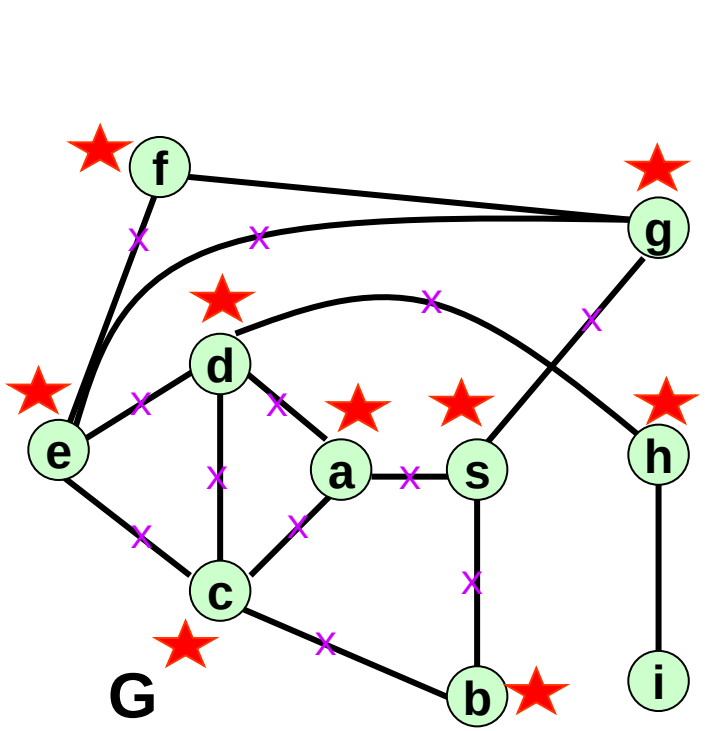


Q



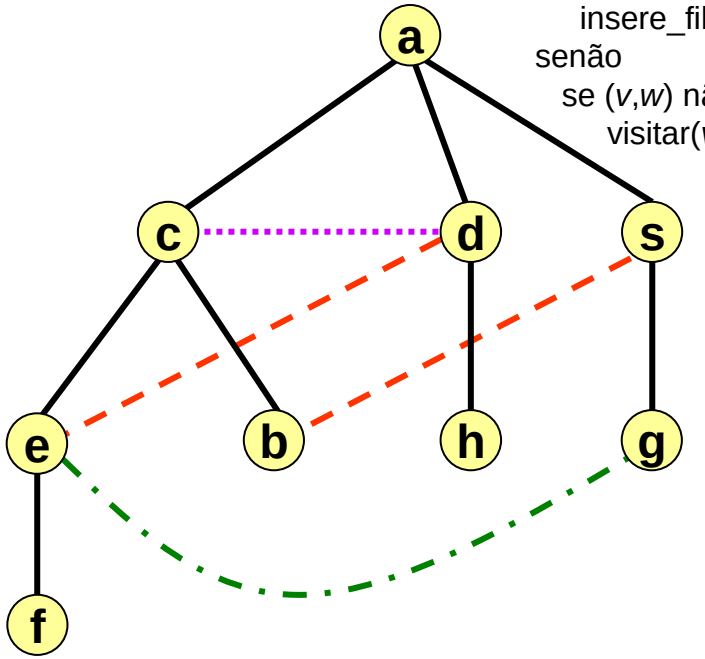
Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



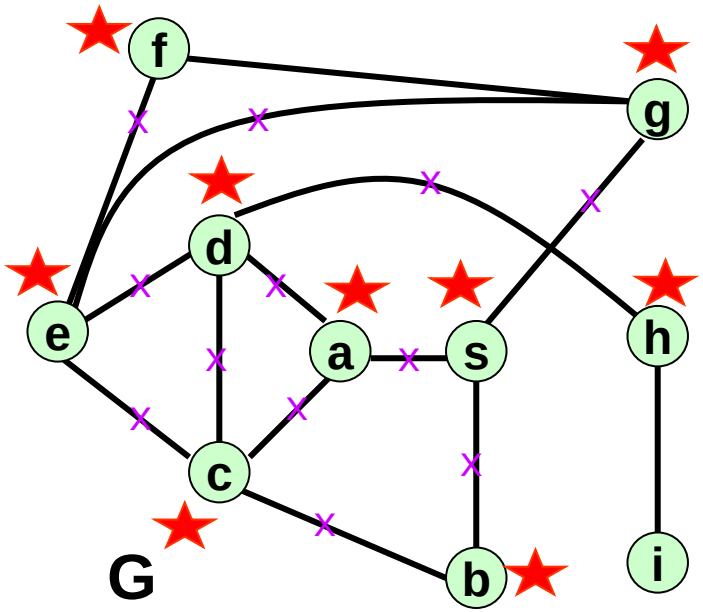


Q



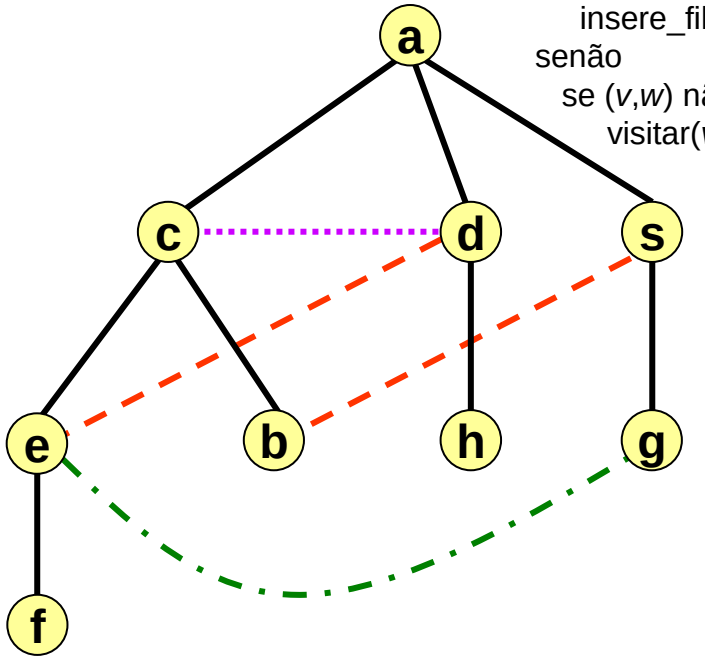
Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```





Q

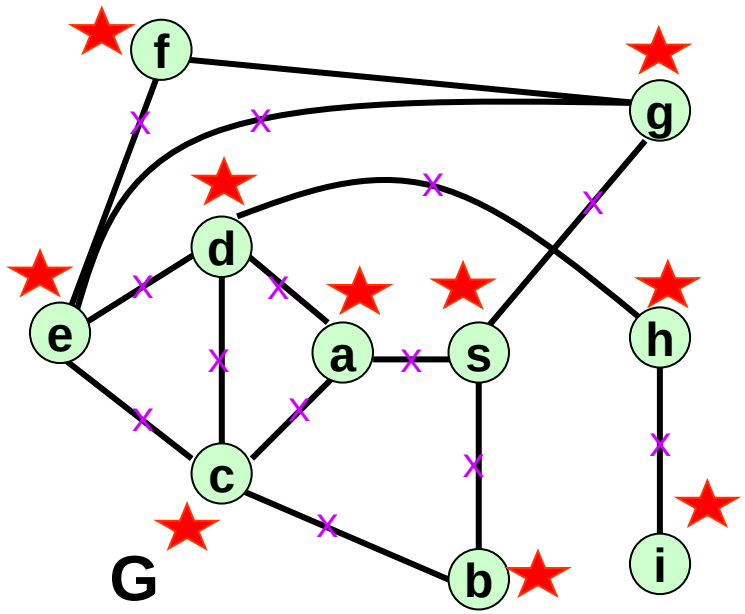


Algoritmo

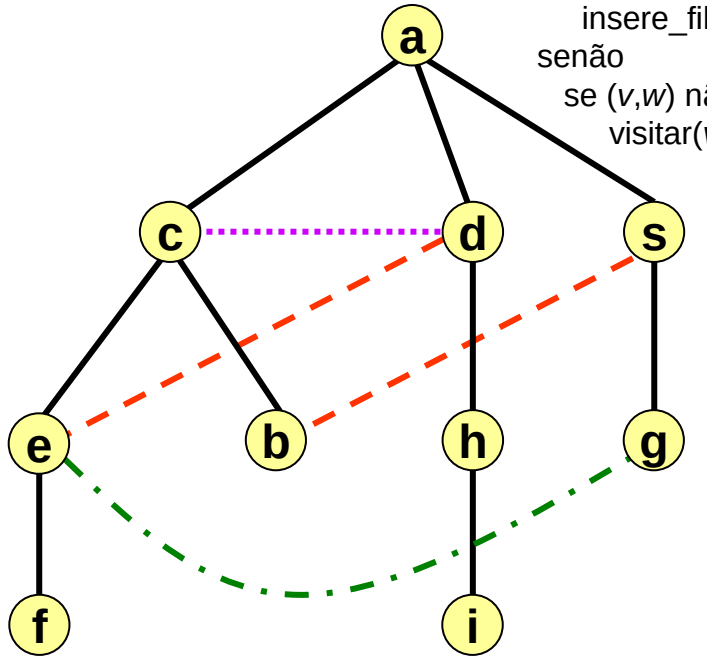
```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



Q



G

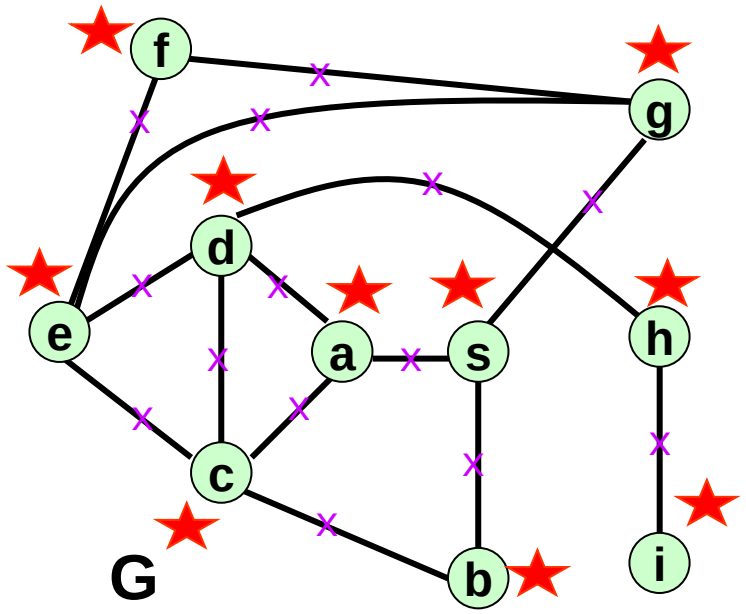


Algoritmo

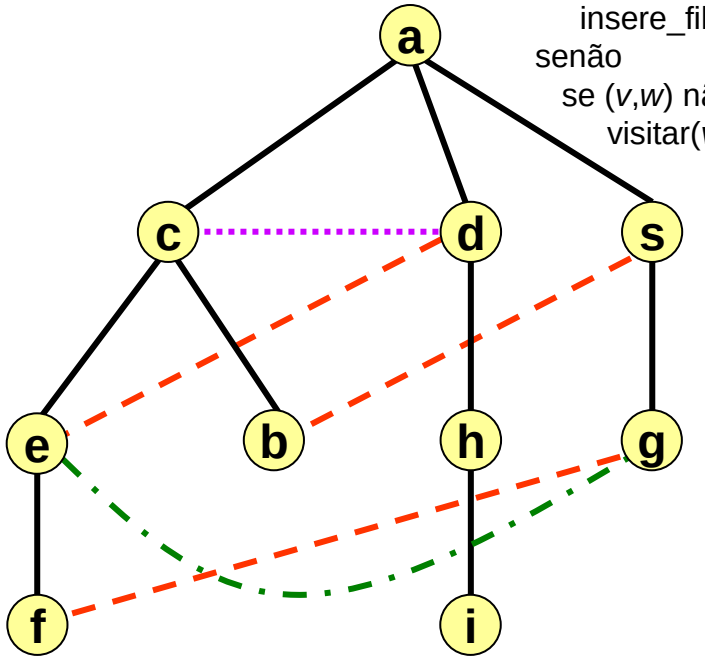
```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



Q



G

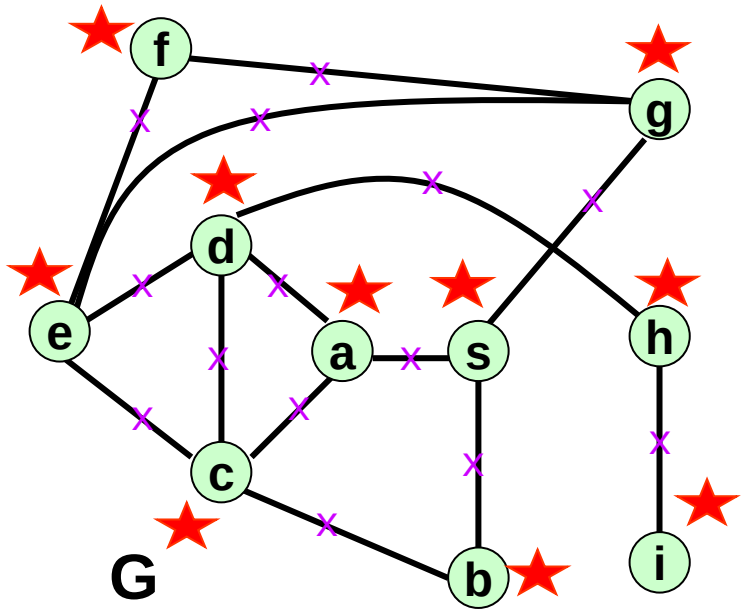


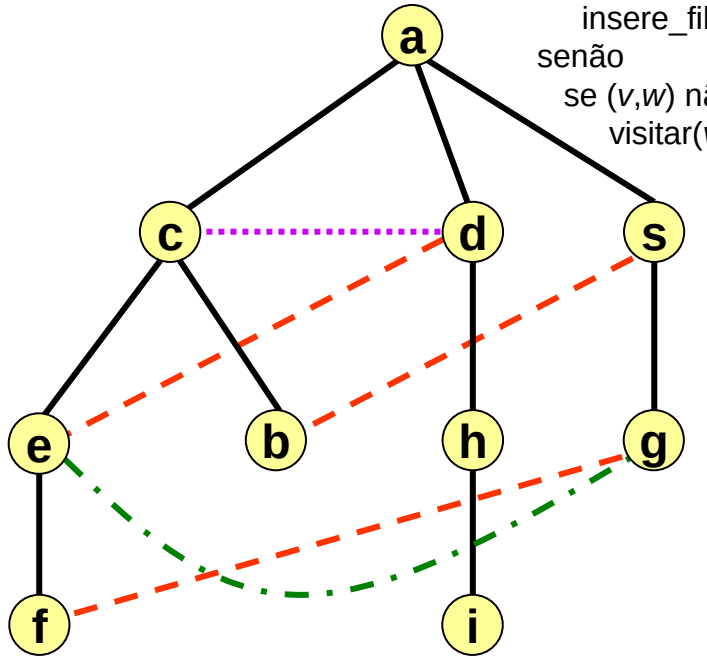
Algoritmo

```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



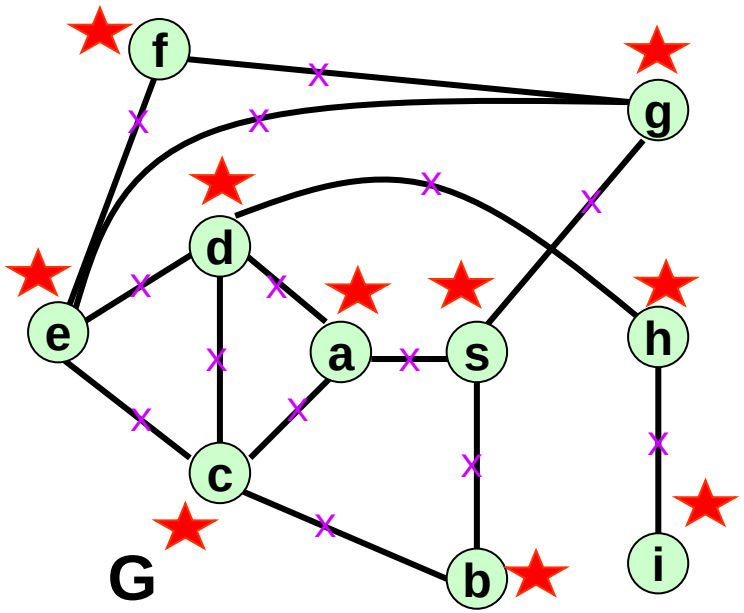
Q



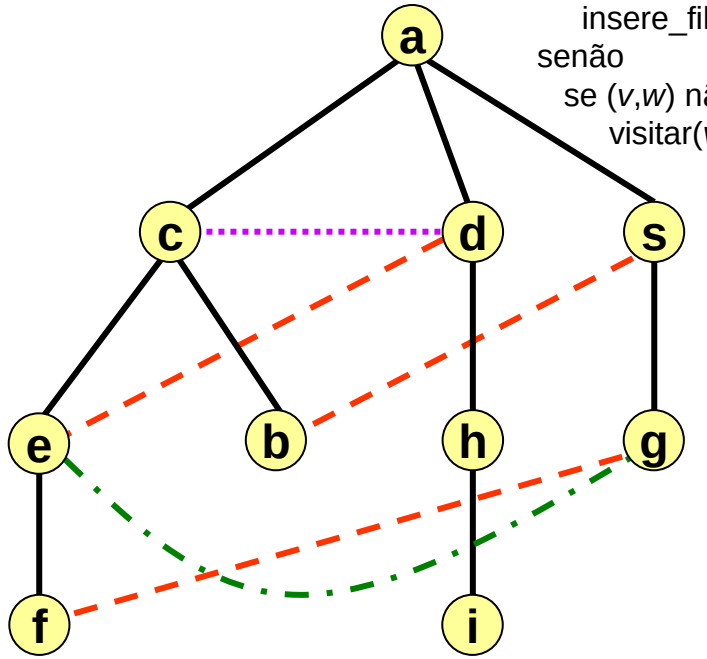


Algoritmo

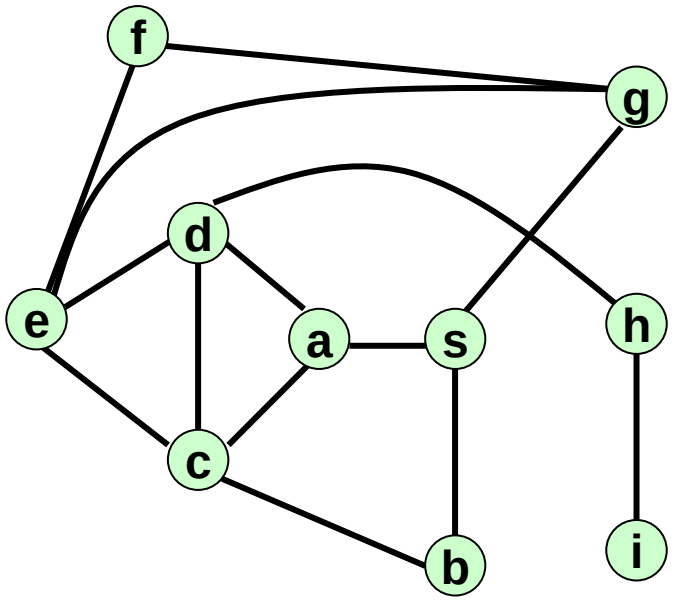
```
Procedimento largura(G=(V,E))
  // A(v) é a lista de vértices adjacentes a v
  Definir uma fila Q
  Definir uma raiz s ∈ V
  Insere_fila(Q,s); marcar(s)
  enquanto Q é não vazia
     v ← remove_fila(Q)
     para w ∈ A(v) efetuar
     se w não é marcado então
          insere_fila(Q,w); marcar(w); visitar(v,w)
     senão
       se (v,w) não visitada
          visitar(v,w)
```



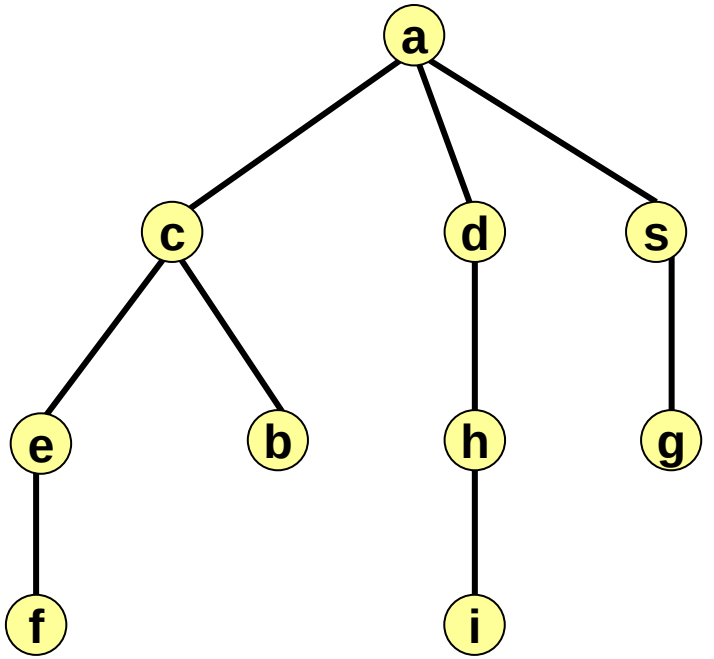




Busca em Largura



Grafo



Árvore de Largura

Busca em Largura

Seja E_T o conjunto das arestas visitadas em (I)

Teorema. O grafo $T(V,E_T)$ é uma árvore geradora de G.

Prova. **EXERCÍCIO**

Teorema. Seja G(V,E) um grafo conexo, $(v,w) \in E$ e $T(V,E_T)$ uma árvore de largura de G. Então nível(v) e nível(w) diferem, no máximo, de uma unidade.

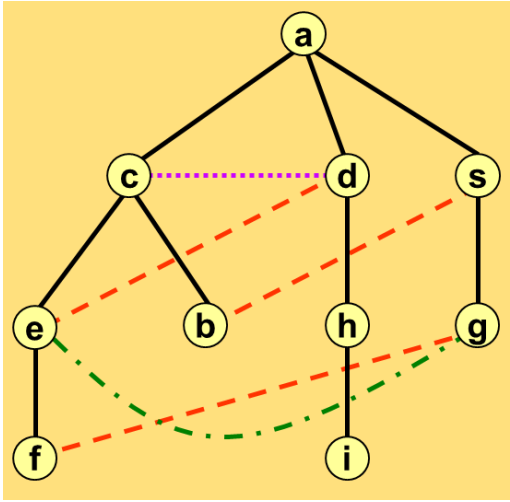
Prova. **EXERCÍCIO**

Busca em Largura

Observe que o teorema anterior divide as arestas (v,w) em duas classes:

1. |nível(v) – nível(w)| = 1

2. |nível(v) – nível(w)| = 0



Cada uma dessas classes pode ser subdividida em outras duas.

Busca em Largura

Considere a aresta (v,w) com nível(v) $\leq$ nível(w)

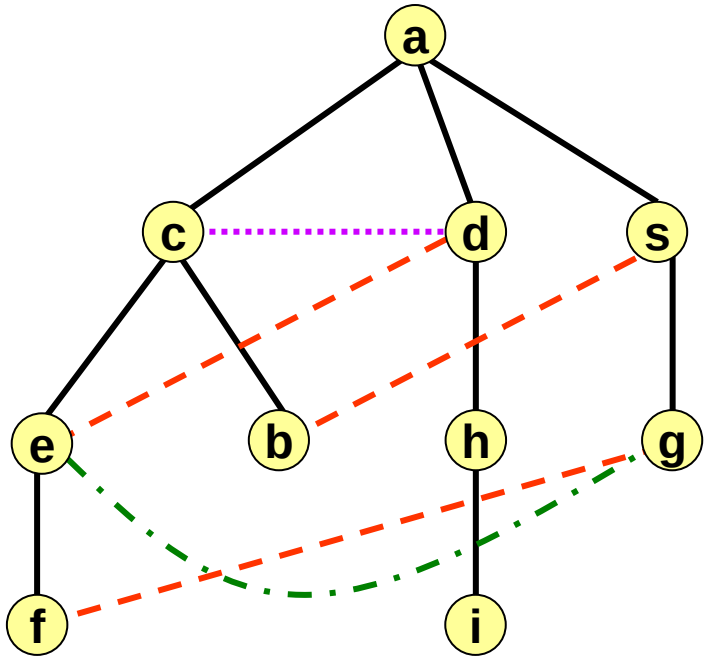
1. (v,w) é aresta ***pai*** (aresta da árvore), quando *v* é pai de *w* em T.

2. (*v*,*w*) é aresta ***tio*** quando nível(w) = nível(v) + 1 e (v,w) $\notin$ T

3. (v,w) é aresta ***irmão*** quando v e w possuem o mesmo pai em T

4. (v,w) é aresta ***primo*** quando nível(v) = nível(w) e v, w não possuem o mesmo pai em T

Busca em Largura



Aresta pai

Aresta irmão

Aresta tio

Aresta primo

Busca em Largura

Para algumas aplicações pode ser importante a ordem em que os vértices foram removidos da fila.

Define-se largura(v) como sendo o número de ordem em que v foi retirado da fila Q.

Busca em Largura

Como usar Busca em Largura para determinar o diâmetro de um grafo?
Qual a complexidade?